and used to authenticate the request.

When a request containing the authentication cookie is received, Kestrel creates the default, unauthenticated, anonymous principal and assigns it to the `HttpContext.User` principal. Any middleware that runs before the `AuthenticationMiddleware` sees the request as unauthenticated, even if there's a valid cookie.

TIP If it looks like your authentication system isn't working, double-check your middleware pipeline. Only middleware that runs after `AuthenticationMiddleware` will see the request as authenticated.

The `AuthenticationMiddleware` is responsible for setting the current user for a request. The middleware calls the authentication services, which reads the cookie from the request, decrypts it, and deserializes it to obtain the `ClaimsPrincipal` created when the user logged in.

The `AuthenticationMiddleware` sets the `HttpContext.User` principal to the new, authenticated principal. All subsequent middleware now knows the user principal for the request and can adjust its behavior accordingly (for example, displaying the user's name on the home page or restricting access to some areas of the app).

NOTE The `AuthenticationMiddleware` is responsible only for authenticating incoming requests and setting the `ClaimsPrincipal` if the request contains an authentication cookie. It is not responsible for redirecting unauthenticated requests to the login page or rejecting unauthorized requests; that is handled by the `AuthorizationMiddleware`, as you'll see in chapter 24.

The process described so far, in which a single app authenticates the user when they log in and sets a cookie that's read on subsequent requests, is common with traditional web apps, but it isn't the only possibility. In chapter 25 we'll take a